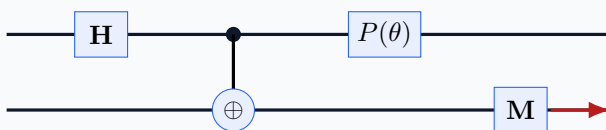


QPU Examples and Troubleshooting

Runnable Protocols, Diagnostics, and Correction Lab



Bailey Forbes
September 13, 2026

Complete examples with line-by-line explanations, common protocol failures, diagnostic messages, truth-table checks, and guided Correction Lab recovery.

Contents

1	How to use the QPU website	1
1.1	Five programming words used in this guide	1
1.2	Interface words and how they relate	1
1.3	Run a bundled example without writing code	1
1.4	Compile a complete protocol	1
1.5	Complete examples versus fragments	2
1.6	When protocol checks report a problem	2
1.7	Saving and reopening work	2
2	First circuit: from source to result	2
2.1	Read the program line by line	2
2.2	What one run means	3
2.3	Adding user-controlled input	3
3	Worked example: reversible half-adder	3
3.1	Classical specification	3
3.2	QPU protocol	3
3.3	What every term is doing	4
3.4	Reversible state equation	4
3.5	Companion truth-table metadata	4
4	Worked example: randomness versus interference	5
4.1	Classical fair coin model	5
4.2	Quantum preparation with Hadamard	5
4.3	Insert phase and recombine paths	5
4.4	Term-by-term explanation	5
4.5	Comparison table	6
5	Process composition and return values	6
5.1	MAIN-PROCESS (lowered/executed)	6
5.2	DECLARECHILD (lowered/executed)	6
5.3	RUNCHILD — expand a child process (lowered/executed)	6
5.4	CALL — alternate child invocation (lowered/executed)	7
5.5	RETURNVALS (lowered/executed)	7
5.6	ACCEPTVALS (lowered/executed)	7
5.7	Nested full-adder example	7
6	Truth-table companion syntax	8
6.1	Text .qpuio structure	8
6.2	MAIN-PROCESS: table header	8
6.3	INPUTS: and OUTPUTS: column groups	8
6.4	Column header row	9

6.5	Indexed data rows	9
6.6	CSV spelling	9
6.7	Partial truth tables	9
6.8	JSON <code>.qpuio</code> envelope	9
6.9	How a table test runs	10
6.10	Protected bundled tables	10
7	Complete worked protocol	10
8	Error conditions and practical rules	11
9	Troubleshooting protocols	11
9.1	Unknown instruction	11
9.2	Missing gate output	11
9.3	Wrong number of controls	12
9.4	Invalid phase angle	12
9.5	Unknown child process	12
9.6	Truth-table columns do not match	12
9.7	Warnings for accepted but inactive syntax	13
9.8	Using the Correction Lab	13
10	Diagnostic message reference	13
11	Glossary	13

1 How to use the QPU website

This chapter assumes no programming experience. You can build circuits by clicking controls, or edit the text protocol directly. Both paths describe the same circuit.

1.1 Five programming words used in this guide

Source text The human-readable instructions in the protocol editor. It is ordinary text, like a note, with one QPU instruction per logical line.

Instruction One action for the QPU system, such as SET, H, CNOT, or MEASURE. The first word says what action to perform.

Argument Extra information after an instruction. In SET Q 0p, Q and 0p are arguments: they say which wire and which state.

Compile Translate source text into the ordered visual gates that the simulator can run. Compilation checks spelling, required inputs, and targets.

Run Apply the compiled gates to the selected starting state. Running changes simulator state; it does not rewrite the source text.

1.2 Interface words and how they relate

Word on screen or in source	Meaning
Particle / qubit	One simulated quantum two-state system. The interface may say particle where circuit theory says qubit.
Wire	The horizontal visual path followed by one qubit through the circuit.
Token	A source-text name, such as Q or Carry, that resolves to a wire.
Register	One or more qubit wires considered as a group.
Gate	A box or controlled symbol that transforms state.
Target	The wire changed by a gate.
Control	A wire whose state decides whether a controlled gate acts.
Protocol / process	The complete source description of a named circuit.

1.3 Run a bundled example without writing code

1. Open the navigation menu and choose **Circuit builder**.
2. Find **Load a starter circuit** and choose an example.
3. Inspect the horizontal wires and gate blocks on the circuit canvas.
4. Choose **Play Sequence** to watch gates advance, or **Run all** to jump to the final state.
5. Choose **Measure all** if the circuit has not already measured its outputs.
6. Read the particle/state display and runtime log. A measured qubit is a classical 0 or 1 outcome for that run.
7. Choose **Reset state** before repeating the same experiment.

1.4 Compile a complete protocol

Listings marked **Complete runnable protocol** may be copied as a whole:

Complete runnable protocol

```
MAIN-PROCESS FirstWebsiteCircuit
CREATETOKEN -I Q
SET Q 0p
H -I Q -O Q
MEASURE -I Q
RETURNVALS Q
```

1. Scroll to **Compile text into a circuit**.
2. Replace the editor contents with the complete listing.
3. Choose **Compile protocol**.
4. Read **Protocol checks**. Zero errors means compilation may proceed. Warnings identify accepted syntax that may not behave as expected.
5. Return to the run controls and choose **Run all**.

Expected result: the H gate prepares equal 0 and 1 amplitudes. MEASURE returns either 0 or 1. Repeating from Reset state should produce both outcomes over many runs.

1.5 Complete examples versus fragments

This documentation uses three listing labels:

Complete runnable protocol Copy the entire block into the protocol editor and compile it.

Instruction fragment A few lines demonstrating one syntax rule. Add them to a complete process rather than compiling the fragment alone.

Companion metadata Truth-table text saved separately as `.qpuio`; do not paste it into the protocol editor.

1.6 When protocol checks report a problem

An **error** blocks successful compilation. A **warning** means the text is accepted but may use inactive, incomplete, or surprising behavior. Each diagnostic gives a line number, the relevant source line when available, and a suggested correction.

Choose **Open Correction Lab for guided help** below the diagnostic list when the correction is not obvious. The Correction Lab can test a protocol, probe outputs, explain failures, and propose gate-level guidance. It does not change protected bundled truth tables.

1.7 Saving and reopening work

Choose **Download as .qpucir** to save protocol source. The `-qpucir.txt` option contains the same source for devices that only handle plain text files. Use the navigation menu's **Upload files** page to reopen a protocol, optionally together with its matching `.qpuio` truth-table metadata.

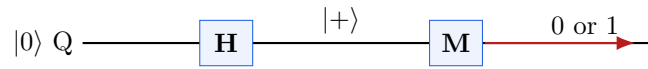
2 First circuit: from source to result

The shortest useful learning path is to prepare one qubit, transform it, measure it, and declare it as an output:

```
MAIN-PROCESS FirstCircuit
CREATETOKEN -I Q
SET Q 0p
H -I Q -0 Q
MEASURE -I Q
RETURNVALS Q
```

2.1 Read the program line by line

1. `MAIN-PROCESS FirstCircuit` gives the protocol a name.
2. `CREATETOKEN -I Q` reserves a logical wire named Q.
3. `SET Q 0p` prepares Q in the basis state $|0\rangle$.
4. `H -I Q -0 Q` applies Hadamard to that same wire, producing $|+\rangle = (|0\rangle + |1\rangle)/\sqrt{2}$.
5. `MEASURE -I Q` samples 0 or 1 and collapses the state.
6. `RETURNVALS Q` declares Q as the first public result.



2.2 What one run means

One execution gives one random-looking sample. For this circuit, theory predicts equal probability for 0 and 1. Getting 0 once does not mean H failed, and getting 1 once does not prove a bias. Repeat the circuit to compare observed frequencies with the predicted distribution.

2.3 Adding user-controlled input

Move a wire into PARAMS when callers or truth-table rows should choose its start state:

```
PARAMS: Input:state

MAIN-PROCESS Inverter
X -I Input -0 Input
MEASURE -I Input
RETURNVALS Input
```

The matching deterministic truth table is:

Input	Output
0p	1p
1p	0p

The rest of this manual explains every symbol in these examples before moving to multi-qubit gates, child processes, and full truth-table files.

3 Worked example: reversible half-adder

A half-adder adds two one-bit numbers without a carry input. Classical theory defines two outputs:

$$\text{Sum} = A \oplus B, \quad \text{Carry} = A \wedge B.$$

The name *half-adder* means it handles half of the full-adder interface: there is no C_{in} input.

3.1 Classical specification

A	B	Carry	Sum	Binary interpretation
0	0	0	0	$0 + 0 = 00_2$
0	1	0	1	$0 + 1 = 01_2$
1	0	0	1	$1 + 0 = 01_2$
1	1	1	0	$1 + 1 = 10_2$

The Carry column is the high-order bit and Sum is the low-order bit. In the last row, decimal $1 + 1 = 2$, written 10_2 in binary.

3.2 QPU protocol

```

PARAMS: A:state B:state

MAIN-PROCESS HalfAdder
CREATETOKEN -I Carry Sum
SET Carry 0p
SET Sum 0p

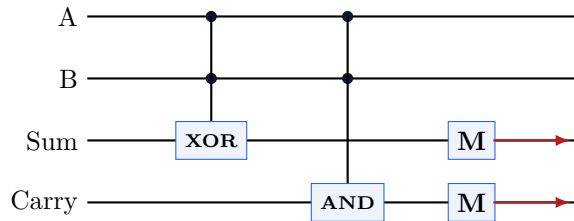
XOR -I A B -O Sum
AND -I A B -O Carry

MEASURE -I Carry
MEASURE -I Sum
RETURNVALS Carry Sum

```

3.3 What every term is doing

1. **PARAMS** declares A and B as caller-controlled state inputs. In a basis-state truth-table test they receive 0p or 1p.
2. **Carry** and **Sum** are **workspace targets**. CREATETOKEN reserves their names; SET prepares both in zero.
3. **XOR** uses A and B as **controls/inputs**. It mutates Sum according to $s' = s \oplus A \oplus B$. Because $s = 0$, the result is $A \oplus B$.
4. **AND** uses the same **preserved controls**. It mutates Carry according to $c' = c \oplus (A \wedge B)$. Because $c = 0$, Carry becomes the classical carry predicate.
5. **MEASURE** produces **classical samples**. For basis inputs this example is deterministic. If A or B is in superposition, outputs may be entangled with the inputs and measurement samples a branch.
6. **RETURNVALS** defines **interface order**. Carry is output column zero and Sum is output column one. It does not perform measurement itself.



3.4 Reversible state equation

Before measurement, the full basis-state action is

$$|A, B, s, c\rangle \mapsto |A, B, s \oplus A \oplus B, c \oplus (A \wedge B)\rangle.$$

The familiar half-adder is the special case $s = c = 0$. Writing the complete equation avoids a common mistake: XOR and AND do not overwrite arbitrary targets. They reversibly combine results with the targets' previous states.

3.5 Companion truth-table metadata

```

MAIN-PROCESS: HalfAdder
INPUTS: A B
OUTPUTS: Carry Sum
# A B Carry Sum
0 0p 0p 0p 0p
1 0p 1p 0p 1p
2 1p 0p 0p 1p
3 1p 1p 1p 0p

```

This file compares the measured QPU outputs with the classical half-adder specification. Passing all four rows demonstrates basis-state equivalence; it does not by itself characterize phases for superposition inputs.

4 Worked example: randomness versus interference

Classical random choice and quantum superposition can produce identical single-measurement frequencies. They are not the same theory. This example uses an intermediate phase change to distinguish them.

4.1 Classical fair coin model

Imagine a classical variable R chosen by a fair coin:

$$P(R = 0) = \frac{1}{2}, \quad P(R = 1) = \frac{1}{2}.$$

Once chosen, R is one definite value even if the observer has not looked. Applying classical NOT changes 0 to 1 and 1 to 0, but the distribution remains half and half. There is no operation that makes the probability of one hidden branch cancel the other branch.

4.2 Quantum preparation with Hadamard

```
MAIN-PROCESS QuantumCoin
CREATETOKEN -I Q
SET Q 0p
H -I Q -0 Q
MEASURE -I Q
RETURNVALS Q
```

Before measurement,

$$|0\rangle \xrightarrow{H} |+\rangle = \frac{|0\rangle + |1\rangle}{\sqrt{2}}.$$

The amplitudes are $1/\sqrt{2}$ and $1/\sqrt{2}$; squaring magnitudes gives the same half-and-half distribution as the classical coin. If measurement happens now, this experiment alone cannot distinguish the two descriptions.

4.3 Insert phase and recombine paths

```
MAIN-PROCESS PhaseInterference
CREATETOKEN -I Q
SET Q 0p
H -I Q -0 Q
Z -I Q -0 Q
H -I Q -0 Q
MEASURE -I Q
RETURNVALS Q
```

Track the state:

$$|0\rangle \xrightarrow{H} \frac{|0\rangle + |1\rangle}{\sqrt{2}} \xrightarrow{Z} \frac{|0\rangle - |1\rangle}{\sqrt{2}} \xrightarrow{H} |1\rangle.$$

4.4 Term-by-term explanation

Superposition After the first H , both basis states have coherent amplitudes. No measurement has selected one.

Phase Z multiplies only the $|1\rangle$ amplitude by $-1 = e^{i\pi}$. Its magnitude remains $1/\sqrt{2}$, so immediate 0/1 probabilities remain equal.

Relative phase The plus relationship between amplitudes becomes a minus relationship. A global minus on both terms would not matter; changing only one term does.

Interference The final H mixes the basis amplitudes. Contributions to the $|0\rangle$ output cancel, while contributions to $|1\rangle$ add.

Measurement The final state is $|1\rangle$, so measurement returns 1 with probability one.

The cancellation can be seen directly from matrix multiplication:

$$H \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ -1 \end{bmatrix} = \frac{1}{2} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix} \begin{bmatrix} 1 \\ -1 \end{bmatrix} = \frac{1}{2} \begin{bmatrix} 0 \\ 2 \end{bmatrix} = \begin{bmatrix} 0 \\ 1 \end{bmatrix}.$$

Classical probability weights are nonnegative and would add rather than cancel. Quantum amplitudes are signed or complex, so phase can create destructive and constructive interference before the Born rule turns amplitudes into probabilities.

4.5 Comparison table

Question	Classical fair coin	H–Z–H quantum circuit
State before observation	One hidden definite bit sampled fairly	Coherent complex amplitude vector
Effect of middle sign/phase	No probability analogue that preserves a fair distribution and later cancels a branch	Z changes relative phase without changing immediate basis probabilities
Can paths cancel?	No; probabilities add	Yes; amplitudes add and may cancel
Final result in this example	Still a distribution unless another classical rule forces a value	Deterministic 1

5 Process composition and return values

5.1 MAIN-PROCESS (lowered/executed)

```
MAIN-PROCESS FullAdder
```

This marks the process body and supplies its catalog/library name. Compilation already begins from the named process, so the row itself emits no gate. The name is the first positional word after **MAIN-PROCESS**; avoid spaces inside it. This name connects a protocol to catalog entries, child calls, downloads, and companion truth-table metadata. If no marker exists, the system uses **InlineProcess**, but named processes are strongly recommended.

5.2 DECLARECHILD (lowered/executed)

```
DECLARECHILD SingleBitFullAdder
```

The named child is bound from the compilation **librarySources** catalog. An unknown name fails on this line. The row itself emits no gate; later **RUNCHILD** or **CALL** expansions use the bound process body. A successful declaration therefore guarantees that the named child exists in the catalog supplied to this compile, and that later calls of that name expand the bound body rather than a later alias.

5.3 RUNCHILD — expand a child process (lowered/executed)

```
RUNCHILD SingleBitFullAdder \
-I A B Cin \
-O Carry Sum
```

The first positional argument is the child process name. Inputs bind by the child's PARAMS order: A maps to the first parameter, B to the second, and Cin to the third in this example. Outputs bind by the child's RETURNVALS order. Before expansion, each bound parent output is zero-prepared once. Child-local names receive a unique scope.

RUNCHILD is compile-time composition, not a dynamic subroutine call in a running quantum machine. The child body is recursively expanded into the parent's flat gate sequence. This allows the renderer and simulator to inspect every resulting gate.

5.4 CALL — alternate child invocation (lowered/executed)

```
CALL SingleBitFullAdder -I A B Cin -O Carry Sum
```

CALL currently follows the same child lookup, parameter binding, output preparation, and expansion path as RUNCHILD. RUNCHILD is used by the bundled multi-bit protocols and makes compile-time expansion explicit; CALL is retained as an equivalent system spelling. Both fail when the named child is absent from the supplied process library.

5.5 RETURNVALS (lowered/executed)

```
RETURNVALS Carry Sum
```

Positional tokens define the process's ordered public outputs. They determine child output binding, truth-table output columns, and the UI's preferred logical width. Do not use -I or -O here: flags would be retained as positional return tokens.

RETURNVALS does not measure automatically. Add MEASURE instructions when classical outcomes are required. Returning an unmeasured wire preserves it as part of the projected quantum output shown by the system.

5.6 ACCEPTVALS (lowered/executed)

```
CALL Child -I A B
ACCEPTVALS LocalCarry LocalSum
```

Each positional local token aliases the corresponding value from the most recent child return list. Direct -O binding on RUNCHILD/CALL is clearer when output names are known.

Ordering is again essential: the first ACCEPTVALS name receives the first returned reference, and so on. The instruction refers to the most recently completed child expansion in compiler state; inserting another child call changes which return list is accepted.

5.7 Nested full-adder example

```
# child.qpucir
PARAMS: A:state B:state Cin:state
MAIN-PROCESS ChildAdder
CREATETOKEN -I Carry Sum
SET Carry Op
SET Sum Op
XOR -I A B -O Sum
CNOT -I Cin -O Sum
CCNOT -I A B -O Carry
CCNOT -I A Cin -O Carry
CCNOT -I B Cin -O Carry
RETURNVALS Carry Sum

# parent.qpucir
PARAMS: A:state B:state Cin:state
MAIN-PROCESS Parent
CREATETOKEN -I Cout Result
RUNCHILD ChildAdder -I A B Cin -O Cout Result
MEASURE -I Cout
```

```
MEASURE -I Result
RETURNVALS Cout Result
```

For basis inputs, the full-adder equations are

$$\text{Sum} = A \oplus B \oplus C_{\text{in}}, \quad C_{\text{out}} = (A \wedge B) \oplus (A \wedge C_{\text{in}}) \oplus (B \wedge C_{\text{in}}).$$

<i>A</i>	<i>B</i>	<i>C_{in}</i>	<i>C_{out}</i>	<i>Sum</i>
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

6 Truth-table companion syntax

Truth tables are metadata paired to a process; they are not `.qpucir` commands. A protocol describes *how* a circuit works; its companion `.qpuio` describes selected input/output expectations.

6.1 Text `.qpuio` structure

A text table may use spaces or commas:

```
MAIN-PROCESS: HalfAdder
INPUTS: A B
OUTPUTS: Carry Sum
# A B Carry Sum
0 0p 0p 0p 0p
1 0p 1p 0p 1p
2 1p 0p 0p 1p
3 1p 1p 1p 0p
```

6.2 MAIN-PROCESS: table header

The colon is required in the table format:

```
MAIN-PROCESS: HalfAdder
```

This name associates metadata with its protocol. Notice the deliberate syntax difference: protocol source writes `MAIN-PROCESS HalfAdder`, while the truth-table header writes `MAIN-PROCESS: HalfAdder`.

6.3 INPUTS: and OUTPUTS: column groups

```
INPUTS: A B
OUTPUTS: Carry Sum
```

These lines divide the columns into values supplied before simulation and values checked after simulation. Their order must match the later column header. `INPUTS` should correspond to state-typed `PARAMS`; `OUTPUTS` should correspond to `RETURNVALS`. `OUTPUTS` must contain at least one name.

Explicit groups remove ambiguity. If they are absent, the loader can derive the split only when a matching protocol is available and the combined `PARAMS` and `RETURNVALS` widths match the table header.

6.4 Column header row

```
# A B Carry Sum
```

The first nonmetadata row beginning with a hash and whitespace or comma is the ordered column header. Here A and B are inputs, followed by Carry and Sum outputs. Unlike an ordinary protocol hash comment, this row has structural meaning inside a .qpuio file.

6.5 Indexed data rows

```
0 0p 0p 0p 0p
1 0p 1p 0p 1p
```

The first cell is a decimal row index. Indices begin at zero and increase by one without gaps. Remaining cells align left-to-right with the column header. Every row must have exactly the expected width, and every state cell must be 0p, 1p, or sp.

For n binary input columns, a complete table contains 2^n rows. The usual ordering interprets input columns most-significant first: for two inputs the sequence is 00, 01, 10, 11. The loader checks sequential row indices, while semantic validation also rejects duplicate input patterns.

6.6 CSV spelling

Commas may replace whitespace consistently:

```
MAIN-PROCESS: HalfAdder
INPUTS: A,B
OUTPUTS: Carry,Sum
# ,A,B,Carry,Sum
0,0p,0p,0p,0p
```

Spaces around commas are trimmed. Do not mix extra prose into a data row; trailing hash comments are allowed only where the table loader can strip them after the data content.

6.7 Partial truth tables

A partial table lists selected cases rather than all 2^n combinations. It must contain at least one row, no duplicate input pattern, and no more than 2^n rows. Partial tables are useful for state-machine transitions or focused regressions, but passing one proves behavior only for the listed cases.

Input count n	Complete row count 2^n	Example
0	1	constant-output process
1	2	0, 1
2	4	00, 01, 10, 11
3	8	000 through 111
4	16	0000 through 1111

6.8 JSON .qpuio envelope

The equivalent machine-oriented JSON form is:

```
{
  "format": "qpuio",
  "version": 1,
  "processName": "HalfAdder",
  "inputColumns": ["A", "B"],
```

```

    "outputColumns": ["Carry", "Sum"],
    "rows": [
      ["0p", "0p", "0p", "0p"],
      ["0p", "1p", "0p", "1p"],
      ["1p", "0p", "0p", "1p"],
      ["1p", "1p", "1p", "0p"]
    ]
  }
}

```

The `format` value must be `qpuio` and `version` must be 1. Column names must be nonempty strings, `outputColumns` must not be empty, and every nested row array must have exactly

$$\text{inputColumns.length} + \text{outputColumns.length}$$

valid cells.

6.9 How a table test runs

For each listed row, the runner:

1. maps input cells to state PARAMS by column name and order;
2. creates the circuit's start-state vector;
3. executes every compiled gate, including child expansions;
4. measures the declared RETURNVALS;
5. compares only those measured outputs with expected output cells.

An expected output `sp` accepts either measured bit. An input `sp` initializes an equal plus-state superposition. Because one probabilistic sample cannot establish a full distribution, tables are strongest for deterministic basis-state behavior.

6.10 Protected bundled tables

The bundled `SingleBitFullAdder`, `TwoBitFullAdder`, `FourBitFullAdder`, and `PhaseDemo` tables are canonical site assets. Do not rewrite them to make an experiment pass. Create a new uploaded or corrected process/table pair when testing different expectations. This keeps examples reproducible and prevents a protocol correction from silently changing its specification.

7 Complete worked protocol

```

PARAMS: A:state B:state Theta:float

MAIN-PROCESS BellAndLogic
CREATETOKEN -I Q0 Q1 AndOut

SET Q0 $A
SET Q1 $B
SET AndOut 0p

# Quantum segment
H -I Q0 -O Q0
CNOT -I Q0 -O Q1
PHASE=pi/4 -I Q1 -O Q1

# Basis-state comparison output
AND -I A B -O AndOut

MEASURE -I Q0
MEASURE -I Q1
MEASURE -I AndOut
RETURNVALS Q0 Q1 AndOut

```

Starting from $|00\rangle$, the H-CNOT segment prepares

$$|00\rangle \xrightarrow{H \otimes I} \frac{|00\rangle + |10\rangle}{\sqrt{2}} \xrightarrow{\text{CNOT}} \frac{|00\rangle + |11\rangle}{\sqrt{2}}.$$

The phase gate then produces

$$\frac{|00\rangle + e^{i\pi/4}|11\rangle}{\sqrt{2}}.$$

The two quantum measurements are correlated, while **AndOut** compares the basis inputs through $A \wedge B$. A deterministic truth table can specify the logic output, but repeated simulation is needed to characterize Bell-state measurement probabilities.

8 Error conditions and practical rules

1. Unknown opcodes fail with **Unknown command**. **RESET** is not a source opcode.
2. Every QPU gate except **MEASURE** requires **-I** and **-O**, then exact registry arity is checked.
3. **PHASE** requires a valid nonzero-denominator angle after **=**.
4. **SWAP** requires two matching input/output wires.
5. **RUNCHILD**/**CALL** fails if the named process is absent from the library. Unknown **DECLARECHILD** names fail on the declaration line.
6. **SET** fails when its positional value is missing.
7. **RETURNVALS** order is semantically significant for child bindings and truth-table columns.
8. Use the same input/output token for single-qubit gates.
9. Initialize derived-gate targets deliberately; their operation is **XOR** into the existing target, not destructive assignment.
10. Treat accepted-but-not-executed instructions as compatibility annotations until their runtime lowering is implemented.

9 Troubleshooting protocols

The protocol editor's **Protocol checks** panel separates errors from warnings. Start with the first error because one malformed instruction can make later messages less useful.

9.1 Unknown instruction

Problem fragment

```
FLIP -I Q -O Q
```

Typical error: Unknown command: FLIP

Why: FLIP is not a QPU instruction. The intended bit-flip gate is X.

Corrected fragment

```
X -I Q -O Q
```

9.2 Missing gate output

Problem fragment

```
H -I Q
```

Typical error: H requires an **-O** output.

Why: a one-qubit gate names the same flowing wire as both input and mutated output.

Corrected fragment

```
H -I Q -O Q
```

9.3 Wrong number of controls

Problem fragment

```
CNOT -I A B -O Target
```

Why: CNOT has one control. A two-control bit flip is CCNOT.

Choose the correction that matches the intent

```
CNOT -I A -O Target # one control
CCNOT -I A B -O Target # two controls
```

9.4 Invalid phase angle

Problem fragment

```
PHASE=ninety -I Q -O Q
```

Why: the angle must be numeric radians, degrees ending in d, or pi notation.

Equivalent corrected fragments

```
PHASE=pi/2 -I Q -O Q
PHASE=90d -I Q -O Q
PHASE=1.5708 -I Q -O Q
```

9.5 Unknown child process

Problem fragment

```
RUNCHILD Adder -I A B Cin -O Carry Sum
```

Typical error: Unknown child process 'Adder'

Why: child source must exist in the process catalog and its MAIN-PROCESS name must match exactly. Load the child protocol first, or change Adder to the catalog's declared name.

9.6 Truth-table columns do not match

If protocol source declares

```
PARAMS: A:state B:state
...
RETURNVALS Carry Sum
```

then companion metadata must keep exactly that input and output order:

```
INPUTS: A B
OUTPUTS: Carry Sum
# A B Carry Sum
```

Renaming, reordering, or omitting a column changes the interface and causes validation errors.

9.7 Warnings for accepted but inactive syntax

JOIN, SPLIT, FREE, DELETETOKEN, COMPILEPROCESS, MASTERVAL, SAVE_STATE, and LOAD_STATE are accepted for compatibility but do not currently change the simulated circuit. The `-$R` flag is also preserved without changing substitution. A warning asks the author to confirm that metadata-only behavior is intentional. DECLARECHILD is not in that list: it binds a catalog child so later RUNCHILD/CALL rows can expand it.

9.8 Using the Correction Lab

1. In the protocol editor, read the reported line and suggestion.
2. Choose **Open Correction Lab for guided help**.
3. Confirm the selected process and its truth table. For a new experiment, use an uploaded or newly created process rather than modifying a protected bundled table.
4. Run tests or probe outputs to identify the first behavioral mismatch.
5. Apply gate-level guidance to protocol source.
6. Return to the Circuit builder, compile again, and confirm that Protocol checks no longer reports an error.

The Correction Lab is appropriate for syntax failures, child-process binding problems, and circuits whose measured outputs disagree with their truth table. It cannot make accepted-but-unimplemented operations execute, and it must not rewrite canonical bundled truth-table expectations.

10 Diagnostic message reference

Code	Meaning	First response
EMPTY_PROTOCOL	No instructions were found.	Add a complete protocol starting with MAIN-PROCESS.
EMPTY_PARAMS	PARAMS has no name:type entries.	Add an entry such as Input:state or remove PARAMS.
MISPLACED_PARAMS	PARAMS is not the first instruction.	Move it before MAIN-PROCESS.
UNKNOWN_INSTRUCTION	The first word is not supported.	Check spelling and the language reference.
INVALID_PHASE	PHASE has an unreadable angle.	Use pi/2, 90d, or numeric radians.
INVALID_GATE_ARITY	Input/control or output count is wrong.	Compare the gate's documented -I/-O counts.
COMPILE_ERROR	Syntax was readable but compilation could not complete.	Inspect child names, SET values, and SWAP bindings.
ACCEPTED_ONLY_OPERATION	The instruction currently emits no simulator behavior.	Replace it with implemented behavior or accept metadata-only use.
INACTIVE_PARAMETER_FLAG	-\$R has no current execution effect.	Use explicit bindings.
INACTIVE_REVERSE_PREFIX	A B-prefixed gate is not being inverted.	Use a self-inverse gate or BPHASE where appropriate.
MISSING_MAIN_PROCESS	InlineProcess fallback will be used.	Add a stable MAIN-PROCESS name.
MISSING_RETURN_VALUES	No public outputs are declared.	Add RETURNVALS in display order.

11 Glossary

Term	Meaning
Amplitude	A complex coefficient of a basis state. Its squared magnitude contributes to measurement probability.
Ancilla	An auxiliary qubit or workspace wire, usually prepared in a known state, used to hold intermediate information in a reversible computation.
Basis state	A state used as a coordinate direction. The computational basis for one qubit is $ 0\rangle, 1\rangle$.
Bloch sphere	A geometric representation of pure one-qubit states after ignoring global phase. North/south are $ 0\rangle/ 1\rangle$; longitude represents relative phase.
Boolean algebra	The mathematics of 0/1 values and operations such as NOT, AND, OR, and XOR.
Child process	A separately named protocol expanded into a parent through RUNCHILD or CALL.
Combinational logic	A classical circuit whose output depends only on current inputs, not stored history.
Control	A wire whose basis condition decides whether a controlled operation acts on its target. The control itself is preserved by the standard controlled gates.
Entanglement	A joint multi-qubit state that cannot be written as a tensor product of independent qubit states.
Fan-out	Sending one classical value to multiple destinations. Unknown quantum states cannot be universally copied; CNOT copies only known basis information onto a zero target and otherwise may entangle.
Gate	A state transformation. Closed-system quantum gates are unitary; measurement and reset have different semantics.
Garbage	Reversible-workspace information no longer wanted as output but still present because it cannot simply be erased.
Global phase	A unit complex multiplier applied to the whole state. It does not change physical measurement predictions.
Ket	Dirac notation $ \psi\rangle$ for a state column vector.
Measurement	Conversion of quantum amplitudes into a sampled classical result, with state collapse and renormalization.
Phase	The angular part of a complex amplitude. Relative phase controls interference even when immediate basis probabilities are unchanged.
Process	A named QPU protocol body with ordered PARAMS and RETURNVALS.
Protocol	Plain-text QPU instructions describing preparation, gates, composition, measurement, and returned references.
Qubit	A normalized two-component complex state $\alpha 0\rangle + \beta 1\rangle$, not merely an unknown classical bit.
Register	One or more logical wires considered together. Current JOIN/SPLIT syntax is accepted but does not yet lower to register operations.
Relative phase	The phase difference between amplitudes. Unlike global phase, it can change later interference and measurement probabilities.
Superposition	A coherent linear combination of basis states. <code>sp</code> prepares the equal plus superposition.
Target	The wire whose amplitudes a gate mutates; normally the first <code>-0</code> reference.
Tensor product	The operation $\otimes$ that combines state spaces. Two one-qubit vectors produce a four-amplitude two-qubit vector.
Token	A source-level logical name resolved to a qubit wire or alias during compilation.
Truth table	Listed input/output expectations for measured RETURNVALS. It describes basis behavior but not arbitrary phase or entanglement.
Uncomputation	Running suitable inverse gates in reverse order to return temporary workspace to a known state without irreversible erasure.
Unitary	A matrix satisfying $U^\dagger U = I$, preserving normalization and having a reversible inverse.